

Wobot AI Assignment

Problem Statement : [click here](#)

Given:

1. MP4 Input format.

Output:

1. Detected Face grouped by frames.

Suggested task:

Part 1

1. Reading the MP4 video
2. From gstreamer, scaling down to 640 x 640 px
3. Write JPEG Encoded frames to file system.

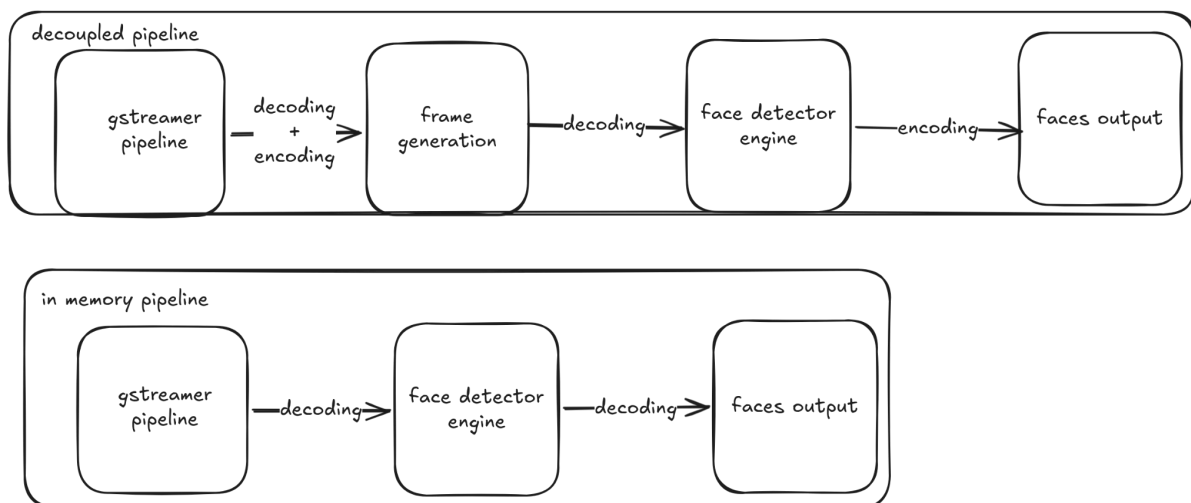
Part 2

1. Read each frame one by one
2. Face detection using OpenCV classifier.
3. Write cropped faces.jpeg grouped by folders.

For the scope of submission, the design of the application is made such that it performs the following:

Part 1 : MP4 → decode → raw frame → JPEG encode → disk write

Part 2: Disk read → JPEG decode → raw frame → detection



This adds unnecessary load on the system.

For Instance,

For a 1 hr video processed at 5fps detection rate:

1. 18,000 jpeg frames generated
2. Assuming each frame is 60kb -> 1.02 gb of intermittent jpeg images.

The CPU spends time encoding and decoding the jpeg images twice which wastes compute and storage both.

A better solution is to directly push the decoded files to the detection pipeline, taking the whole process in memory where none of the compute is redundant and the storage system is untouched.

I have implemented both the workflows for better understanding and evaluation. A comparison chart can be seen below:

Metric	Decoupled Pipeline	In-Memory Pipeline
Execution Time (s)	11.475177284	6.837557369
Intermediate Disk I/O (MB)	3.37	0.00
Total Faces Detected	63	69

Things noted for correctness:

1. For scaling down to 640 x 640, I have essentially added padding to preserve the aspect ratio for accuracy in detection.

Deliverables:

1. Gstreamer script using gstreamer-launch
2. Complete implementation of both Decoupled and In-memory detector pipelines.
3. Unit tests for the pipeline implementations checking dependencies and input validation.
4. Benchmark script for both modes with benchmark_result.md.
5. The frames have been limited to be analysed at 3fps for face detection.

Compilation steps:

1. Decoupled_detector

```
mkdir build
cd build
cmake ..
make
Ctest -V
```

2. In_memory_detector

```
mkdir build
cd build
cmake ..
make
Ctest -V
```

[README.md](#) contains the compilation steps in the repository.

Future SCOPE of integration:

1. Parallel stream of CUDA threads for handling detection using OpenCV CUDA library.
2. For offline processing of GBs of data, we need to present Distributed orchestration layer using docker and caching. It adds more complexity, but at the end, saves time and achieves specific business goals.
3. Real time processing needs to be validated. I have only had the time to think through the possibilities.

Github : <https://github.com/iam-rishabh/face-detection>

Bibliography:

1. <https://www.it-jim.com/blog/gstreamer-for-computer-vision-and-audio-processing/>
2. <https://www.it-jim.com/blog/gstreamer-cpp-tutorial/>
3. https://github.com/agrechnev/gst_app_tutorial